difference between basic object and object in ruby

The fundamental difference between `BasicObject` and `Object` in Ruby is **their place in the inheritance hierarchy and the number of built-in methods they provide**. `BasicObject` **is the absolute root class** of the entire Ruby object hierarchy and is intentionally a "blank class" with almost no methods, while `Object` **is the default root class** for standard user-defined classes and inherits from `BasicObject`. Stack Overflow +4

Because `Object` mixes in the `Kernel` module, it comes packed with dozens of standard utility methods (like `puts`, `inspect`, `nil?`, and `class`) that are completely missing from `BasicObject`. Stack Overflow +2

Key Differences At A Glance

Feature	`BasicObject`	`Object`
Hierarchy Position	Root of all classes (`superclass` is `nil`).	Inherits directly from `BasicObject`.
Kernel Module	Not included.	Included/Mixed in.
Number of Methods	Very few (~8 to 10 core methods).	Many (~50+ default utility methods).
Constant Lookup	Does not automatically see top-level constants.	Resolves top-level constants smoothly.
Primary Use Case	Proxy objects, decorators, and custom DSLs.	Standard application logic and custom classes.

1. Method Availability and the Kernel Module

The starkest contrast between the two lies in what actions they can perform out of the box:

- **`Object`**: When you create a standard class, it automatically inherits from `Object`. `Object` includes the `Kernel` module. This is why methods like `is_a?`, `respond_to?`, `to_s`, and globally available functions like `puts` or `require` work seamlessly everywhere. Stack Overflow +4
- **`BasicObject`**: It is explicitly designed to minimize namespace pollution. It only responds to a handful of absolute essential operations, such as `==`, `equal?`, `__send__`, `__id__`, `instance_eval`, and `instance_exec`. If you try to run `puts` or check `.class` on a `BasicObject` instance, Ruby will raise a `NoMethodError`. Stack Overflow +1

2. Constant Resolution Namespace

The way Ruby looks up constants changes depending on which class you subclass:

- **In `Object`**: If you reference a top-level standard library class like `File` or `Hash` inside a standard class, Ruby finds it immediately because `Object` sits in the standard namespace. Ruby Programming Langua...
- **In `BasicObject`**: It sits completely outside the standard library namespace. If you create a class inheriting from `BasicObject`, you cannot simply type `File` or `Hash`. You must use the absolute global prefix (`::File` or `::Hash`) to reference them. GitHub

3. When to Use Which

99% of the time, developers should use or inherit from `Object` (which is the default when writing `class MyClass; end`). Dev Genius +1

However, `BasicObject` is invaluable for specific metaprogramming patterns:

- **Proxy Objects & Decorators:** When building a proxy class (like Ruby's built-in `Delegator` class) that wraps another object and intercepts all incoming method calls via `method_missing`. If the proxy inherited from `Object`, methods like `to_s` or `inspect` would execute on the proxy itself rather than being passed down to the wrapped object. Stack Overflow +3
- **Domain-Specific Languages (DSLs):** When constructing clean layout execution layers or configuration tools where you want to prevent any built-in Ruby methods from colliding with your custom domain keywords. Stack Overflow +1

Would you like to see a practical **code example** of how a proxy object breaks using `Object` but works perfectly using `BasicObject`?

14 sites

What is the difference between type of a class and superclass ...
What is the difference between type of a class and superclass of a class in ruby? ... I am but confused right no...
Stack Overflow

What's the difference between Object and BasicObject in Ruby?
In Ruby, BasicObject is the parent class of all classes. BasicObject was introduced in Ruby 1.9. Object mixes in th...
Stack Overflow

Ruby OOP Model 1: BasicObject, Object, Kernel - Dev Genius
Having a class called Object is already a bit confusing (but don't worry, later we'll see ...
Dev Genius

Show all

I am unable to understand why do we even require BasicObject class? What can it do which Object was not abl...

The fundamental reason Ruby requires `BasicObject` comes down to Ruby's extreme commitment to **dynamic metaprogramming** and a feature called `method_missing`. Ruby-doc +4

In other languages like Java, C#, or Python, you rarely create an object whose sole purpose is to pretend to be a completely different object at runtime. In Ruby, doing this (called the **Proxy Pattern**) is an everyday practice. Stack Overflow +4

Before `BasicObject` was introduced in Ruby 1.9, developers ran into a major architectural roadblock with the standard `Object` class. ernie.io +2

The Problem: "Method Pollution" and `method_missing`

In Ruby, if you call a method that doesn't exist on an object, Ruby automatically triggers a fallback method called `method_missing`. Developers use this to build **Proxy/Decorator objects** that catch any method call and forward it to something else (like a database record, a remote API, or a hidden object). Ruby-doc +4

For this to work perfectly, **the Proxy object must be a blank slate**. It shouldn't know *how* to do anything itself, so it catches everything in `method_missing` and passes it along. ernie.io +2

The Breakdown with `Object`

Standard `Object` has over 50 built-in methods (via the `Kernel` module) like `to_s`, `inspect`, `clone`, `hash`, and `class`. Dev Genius +2

If your proxy inherits from `Object` and you call `proxy.to_s`, **`method_missing` will never trigger**. The proxy will intercept and execute its *own* `to_s` method instead of forwarding it to the target object. This creates silent, incredibly frustrating bugs where the proxy "leaks" its own identity. Ruby-doc +1

Why didn't other languages encounter this?

Other languages didn't need to create a `BasicObject` because they handle these problems with entirely different language mechanics:

1. **Static Typing & Interfaces (Java/C#):** In Java, you cannot just call an arbitrary method and catch it at runtime with something like `method_missing`. Proxies are built using compile-time interfaces, explicit `java.lang.reflect.Proxy` APIs, or bytecode manipulation. The number of methods on `java.lang.Object` doesn't break proxies because the compiler forces strict structural adherence.
2. **Explicit Dunder Methods (Python):** Python has an `Object` root, but magic methods (like `__str__` or `__repr__`) bypass normal attribute lookups. Python developers use explicit magic methods (`__getattr__`) to build proxies, which behave fundamentally differently than Ruby's message-passing architecture.

The Historic Ruby Patch (BlankSlate)

Before Ruby 1.9 natively provided `BasicObject`, the problem was so bad that prominent libraries like Ruby on Rails and Builder had to hack their own solutions. They created a famous custom class called `BlankSlate`. Its sole purpose was to use loop loops to aggressively erase/undefine every method from standard `Object`: simonecarletti.com +2

```ruby
# How Ruby developers had to fix Object before BasicObject existed:
class BlankSlate
  instance_methods.each do |method|
    undef_method(method) unless method =~ /^__|object_id/
  end
end
```

Use code with caution.

Because the community was constantly hacking `Object` to strip it bare, Matz (the creator of Ruby) officially introduced `BasicObject` as a completely clean, native foundation layer for meta-programming. ernie.io +1

Code Example: The Difference in Action

See exactly how standard `Object` fails where `BasicObject` succeeds when building a simple logger proxy.

```ruby
# Imagine this is the target object we are wrapping
class SecretAgent
  def name; "James Bond"; end
  def to_s; "Agent 007"; end
end

# 1. FAILING PROXY (Inherits from Object by default)
class BadProxy
  def initialize(target)
    @target = target
  end

  # Intercept and log everything
  def method_missing(name, *args, &block)
    puts "LOG: Called #{name}"
    @target.send(name, *args, &block)
  end
end

# 2. SUCCESSFUL PROXY (Inherits from BasicObject)
class GoodProxy < BasicObject
  def initialize(target)
    @target = target
  end

  def method_missing(name, *args, &block)
    # Note: Must reference ::Kernel to use 'puts' inside a BasicObject!
    ::Kernel.puts "LOG: Called #{name}"
    @target.__send__(name, *args, &block)
  end
end

agent = SecretAgent.new

#--- Testing the Bad Proxy ---
bad = BadProxy.new(agent)
bad.name  # Output: LOG: Called name -> "James Bond" (Works!)
bad.to_s  # Output: "#<BadProxy:0x0000...>"  (FAILED: Did not log, did not forward!)

#--- Testing the Good Proxy ---
good = GoodProxy.new(agent)
good.name # Output: LOG: Called name -> "James Bond" (Works!)
good.to_s # Output: LOG: Called to_s -> "Agent 007"   (SUCCESS!)
```

Use code with caution.

Summary

Ruby required `BasicObject` because it is an **ecosystem built heavily on transparent wrappers**. Without a truly empty class, it is impossible to build a bulletproof wrapper that seamlessly catches and delegates 100% of incoming system messages. Reddit · ruby +2

Would you like to explore how **Ruby on Rails** uses this specific behavior under the hood for features like database associations or lazy-loading?

22 sites

What's the difference between Object and BasicObject in Ruby?
17 Jan 2012 — In Ruby, BasicObject is the parent class of all classes. BasicObject was introduced in Ruby 1.9. Object...
Stack Overflow

Ruby OOP Model 1: BasicObject, Object, Kernel - Dev Genius
21 Nov 2024 — Having a class called Object is already a bit confusing (but don't worry...
Dev Genius

Ruby 1.9, BasicObject, and ! (not) - Ernie Miller
10 Apr 2010 — BasicObject to the rescue! Now, let's have a look at the situation in Ruby 1.9. ruby-head > 1.send(:!) =>...
ernie.io

Show all